TypeScript

TypeScript - это строго типизированный и компилируемый язык. На выходе компилятор создает JavaScript, который затем исполняется браузером.

TypeScript реализует многие концепции, которые свойственны объектно-ориентированным языкам, как, например, наследование, полиморфизм, инкапсуляция и так далее.

Потенциал TypeScript позволяет быстрее и проще писать большие сложные комплексные программы, соответственно их легче поддерживать, развивать, масштабировать и тестировать, чем на стандартном JavaScript.

TypeScript развивается как opensource-проект и, как и многие проекты, хостится на гитхабе. Адрес репозитория - https://github.com/Microsoft/TypeScript. Кроме того, он является кроссплатформенным, а это значит, что для разработки мы можем использовать как Windows, так и MasOS или Linux.

Установка

Компилятор TS можно установить с помощью команды менеджера пакетов npm, который используется в Node.js:

npm install -g typescript

Обновить до последней версии:

npm update -g typescript

Проверка версии:

tsc -v

Компиляция

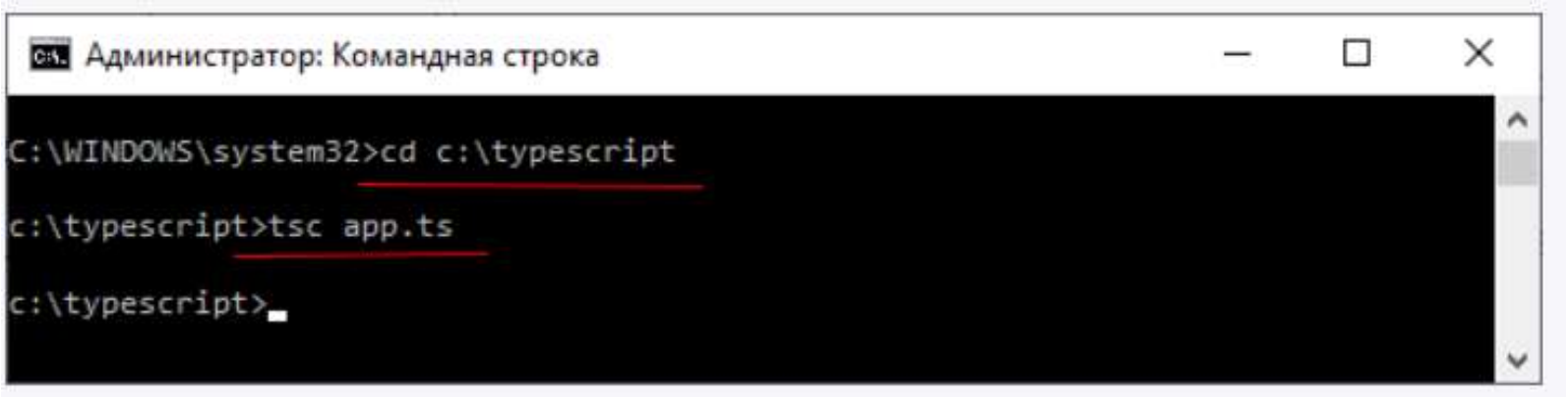
app.ts

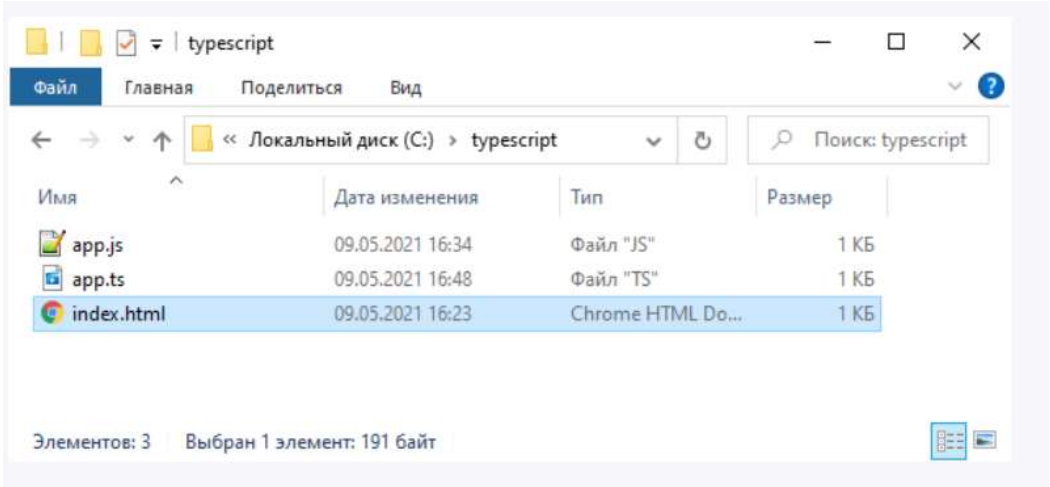
```
class User{
    name : string;
    constructor(_name:string){

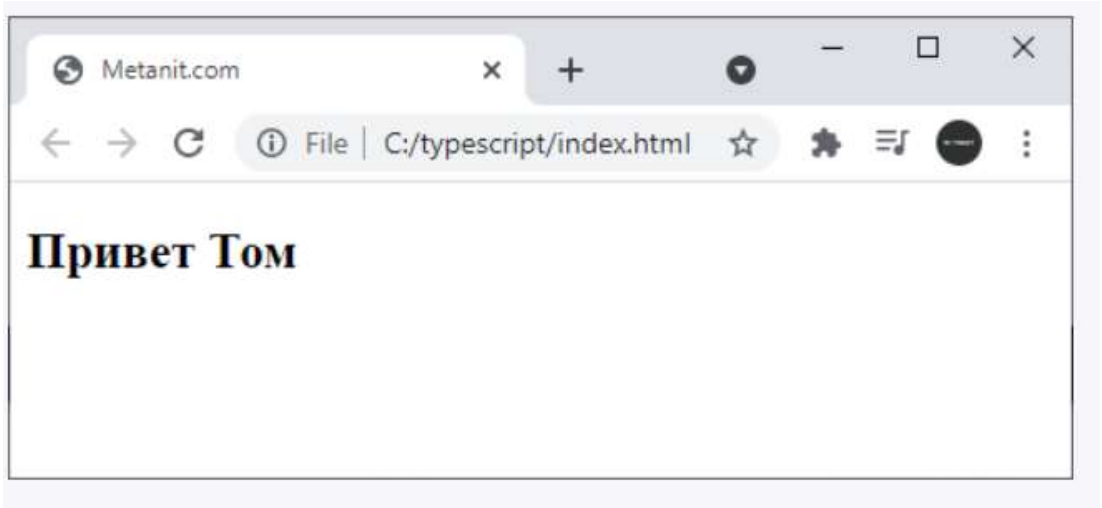
        this.name = _name;
    }
}
const tom : User = new User("Том");
const header = this.document.getElementById("header");
header.innerHTML = "Привет " + tom.name;
```

app.js

```
var User = /** @class */ (function () {
    function User(_name) {
        this.name = _name;
    }
    return User;
}());
var tom = new User("Том");
var header = this.document.getElementById("header");
header.innerHTML = "Привет " + tom.name;
```








```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Metanit.com</title>
</head>
<body>
    <h2 id="header"></h2>
    <script src="app.js"></script>
</body>
</html>
```

Настройки компиляции

Автоматическая перекомпиляция

Опция **--watch**, а также ее сокращенная версия **-w** автоматически перекомпилирует файлы typescript, если в них были внесены какие-либо изменения.

```
tsc -w app.ts
```

Версия ECMAScript

С помощью параметра **–-target** или его сокращенной версии **–t** можно задать версию стандарта JavaScript, в которую будет компилироваться код TypeScript. Этот параметр может принимать следующие значения: "ES3" (по умолчанию), "ES5", "ES6" / "ES2015", "ES7" / "ES2016", "ES2017", "ES2018", "ES2019", "ES2020" или "ESNext":

```
tsc app.ts -t ES6
```

Удаление комментариев

По умолчанию в файлы javascript переходят все комментариии, которыми снабжен код в файлах TS. Удаление комментариев при компиляции осуществляется с помощью параметра –-removeComments:

```
tsc app.ts –removeComments
```

Установка каталога
С помощью параметра --outDir можно задать папку для хранения скомпилированных файлов js:

```
tsc --outDir D:\ts\js app.ts
```

Объединение файлов
Если у нас несколько файлов TS, то с помощью параметра --outFile их можно объединить в один файл js:

```
tsc --outFile output.js app.ts hello.ts
```

Тип модуля
С помощью параметра --module, либо -m можно указать тип модуля, который будет использоваться для компиляции. Эта опция может принимать следующие значения: "None", "CommonJS" (значение по умолчанию, если задана версия ECMAScript "ES3" или "ES5"), "AMD", "System", "UMD", "ES2015", "ES2020" и "ESNext".

```
tsc -m commonjs app.ts
```

Несколько параметров

Если надо задать несколько параметров, то они и их значения последовательно перечисляются через пробел.

```
tsc -t ES5 --outDir js -m commonjs app.ts
```

Вызов справки

И чтобы посмотреть все доступные параметры и справку по ним, можно воспользоваться параметром -h:

```
tsc -h
```

Файл конфигурации tsconfig.json

```
{
   "compilerOptions": {          // параметры компиляции
      "target": "es5",
      "removeComments": true,
      "outDir": "js",
      "sourceMap": true
      "outFile": "main.js"
   },
   "files":[                 // набор включаемых в проект файлов
      "app.ts",
      "interfaces.ts",
      "classes.ts",
   ],
   "exclude":[          // исключает при компиляции определенные файлы
      "wwwroot",
      "node_modules"
   ],
   "compileOnSave": true       // генерировать все файлы js при каждом сохранении файлов TypeScript
}
```

Переменные и константы

С помощью var можно определить два и более раз переменную с одним и тем же именем:

```
var x = "hello";
console.log(x);
var x = "work";
console.log(x);
```

Подобную проблему позволяет решить let:

```
let x = "hello";
console.log(x);
let x = "work"; // здесь будет ошибка, так как переменная x уже объявлена
console.log(x);
```

```
let x = "hello";
console.log(x);
var x = "work"; // здесь будет ошибка, так как переменная x уже объявлена
console.log(x);
```

Для определение констант используется ключевое слово const:

```
const z = 6;
```

Типы данных

В TypeScript имеются следующие базовые типы:

- **boolean**: логическое значение true или false
- **number**: числовое значение
- **string**: строки
- **Array**: массивы
- **кортежи**
- **Enum**: перечисления
- **Any**: произвольный тип
- **Symbol**
- **null и undefined**: соответствуют значениям null и undefined в javascript
- **Never**: также представляет отсутствие значения и используется в качестве возвращаемого типа функций, которые генерируют или возвращают ошибку

Для установки типа применяется знак двоеточия, после которого указывается название типа:

```
let x: number = 10;
let hello: string = "hello world";
let isValid: boolean = true;
```

Тип any

Any описывает данные, тип которых может быть неизвестен на момент написания приложения. Данный код скомпилируется без ошибок:

```
let someVar: any = "hello";
console.log(someVar);   // сейчас someVar - это string
someVar = 20;
console.log(someVar);   // сейчас someVar - это number

var someArray: any[] = [ 24, "Tom", false];



let x;  // тип any
x = 10;
x = "hello";
```

Оператор **typeof** может возвращать следующие значения:

- "string"
- "number"
- "bigint"
- "boolean"
- "symbol"
- "undefined"
- "object"
- "function"

Функции

Можно определить тип передаваемых параметров и тип возвращаемого значения.

```
function add(a: number, b: number): number {
   return a + b;
}
let result = add(1, 2);
console.log(result);
```

При вызове функции
add("1", "2");
компилятор TS выдаст ошибку

Если функция ничего не возвращает, то указывается тип void:

```
function add(a: number, b: number): void {
   console.log(a + b);
}
add(10, 20);
```

При вызове в функцию должно передаваться ровно столько значений, сколько в ней определено параметров!

Необязательные параметры должны быть помечены вопросительным знаком ?. Необязательные параметры должны идти после обязательных:

```
function getName(firstName: string, lastName?: string) {
    if (lastName)
        return firstName + " " + lastName;
    else
        return firstName;
}

let name1 = getName("Иван", "Кузнецов");
console.log(name1); // Иван Кузнецов
let name2 = getName("Вася");
console.log(name2); // Вася
```

Тип функции

Тип функции представляет комбинацию типов параметров и типа возвращаемого значения.

(параметр1: тип, параметр2: тип,...параметрN: тип) => тип_результата;

Можно определять значения типа функции:

```
function sum (x: number, y: number): number {
    return x + y;
};
function subtract (a: number, b: number): number {
    return a - b;
};

let op: (x:number, y:number) => number;

op = sum;
console.log(op(2, 4));  // 6

op = subtract;
console.log(op(6, 4));  // 2
```

Если определенный тип функции надо очень часто использовать, то для него можно определить псевдоним и обращаться к типу по этому псевдониму:

```
type Operation = (a: number, b: number) => number;

function mathOp(x: number, y: number, op: Operation): number{

    return op(x, y);
}
let sum: Operation = function(x: number, y: number): number {
    return x + y;
};

console.log(mathOp(10, 20, sum)); // 30
```

Объединения union

Объединения или union не являются типом данных, но они позволяют комбинировать или объединить другие типы. Можно определить переменную, которая может хранить значение двух или более типов:

```
let id : number | string;
id = "1345dgg5";
console.log(id); // 1345dgg5
id = 234;
console.log(id);  // 234
```

Можно использовать проверку на тип, чтобы разграничить логику для различных типов:

```
function printSentence(words: string[]|string){
    // если words - строка
    if (typeof words === "string") {
      console.log(words);
    } else {
      // Если words - массив string[]
      console.log(words.join(" "));
    }
}
printSentence(["Язык", "программирования", "TypeScript"]);
printSentence("Язык программирования JavaScript");
```

Оператор **!**

Оператор ! (non-null assertion operator) позволяет указать, что объект не представляет значение null и undefined. Так, возьмем следующий пример:

```
const header: HTMLElement|null = document.getElementById("header");
header!.innerText = "Hello Typescript!";
```

Объекты

Во второй строке получим ошибку, поскольку компилятор после первой строки предполагает, что объект person будет тип { name: string; age: number }:

```
let person = { name: "Tom", age: 23 };
person = { name: "Bob" };          // ! Ошибка
```

Поэтому данной переменной можно присвоить другой объект, который соответствует типу { name: string; age: number } по названиям, количеству и типу свойств:

```
let person: { name: string; age: number } = { name: "Tom", age: 23 };
```

```
let person = { name: "Tom", age: 23 };
person = { name: "Bob", age: 35 };
```

Необязательные свойства

```
let person: { name: string; age?: number }; // Свойство age - необязательное

person = { name: "Tom", age: 23 };
console.log(person.name);   // Tom
person = { name: "Bob"};    // Норм, свойство age - необязательное
console.log(person.name);   // Bob
```

Функция может принимать объекты в качестве параметров и могут возвращать объект. В этих случаях необходимо указать тип объекта для параметров и результата функции:

```
function printUser(user: { name: string; age: number}) {
  console.log(`name: ${user.name}  age: ${user.age}`);
}
let tom = {age: 36, name: "Tom"};

printUser(tom);
```

Объект как результат функции:

```
function defaultUser(): { name: string; age: number} {

  return {name: "Tom", age: 37};
}

let user = defaultUser();
console.log(`name: ${user.name}  age: ${user.age}`);
```

Псевдонимы типов

TypeScript позволяет определять псевдонимы типов с помощью ключевого слова type:

```
type id = number | string;

let userId : id = 2;
console.log(`Id: ${userId}`);
userId = "qwerty";
console.log(`Id: ${userId}`);
```

Расширение псевдонимов

Одни псевдонимы могут заимствовать или расширять код других. Для этого применяется операция &:

```
type Person = {name: string; age: number};
type Employee = Person & {company: string};
```

Преобразование к типу

Type assertion представляет модель преобразования значения переменной к определенному типу

Первый способ преобразования заключается в использовании угловых скобок (тип, к которому надо выполнить приведение):

```
const header = <HTMLElement>document.getElementById("header");
header.innerText = "Hello Typescript!";
```

Второй способ заключается в применении оператора as, после которого указывается тип, к которому надо выполнить преобразование:

```
const header = document.getElementById("header") as HTMLElement;
header.innerText = "Hello Typescript!";
```

Массивы

```
let list: number[] = [10, 20, 30];
let colors: string[] = ["red", "green", "blue"];

let names: Array<string> = ["Tom", "Bob", "Alice"];
```

TypeScript позволяет определять массивы, элементы которых нельзя изменять, добавлять новые или удалять. Для этого применяется тип ReadonlyArray<>, для которого в угловых скобках указывается тип элементов массива.

В отличие от типа Array для типа ReadonlyArray нельзя принимать конструктор, как в следующем случае:

```
const people: ReadonlyArray<string> = new ReadonlyArray("Tom", "Bob", "Sam");
```

Также можно использовать сокращение типа - readonly Тип_данных[]:

```
const people: readonly string[]= ["Tom", "Bob", "Sam"];
```

Кортежи

Кортежи (Tuples) представляют набор элементов, для которых уже заранее известен тип. В отличие от массивов кортежи могут хранить значения разных типов:

```
// определение кортежа - кортеж состоит из двух элементов - строки и числа
let user: [string, number];
user = ["Tom", 28];

userInfo = [28, "Tom"]; // Ошибка
```

С помощью оператора **...** внутри определения типа кортежа можно определить набор элементов, количество которых неопределено:

```
let math: [string, ...number[]] = ["Math", 5, 4, 5, 4, 4];
let physics: [...number[], string] = [5, 5, 5, "Physics"];
let chemistry: [string, ...number[], boolean] = ["Chemistry", 3, 3, 4, 5, false];
```

Кортежи только для чтения:

```
const tom: readonly [string, number] = ["Tom", 36];
tom[1] = 37; // ! Ошибка - элементы кортежа для чтения нельзя изменять
```

Перечисление enum

Тип enum или перечисление позволяет определить набор именнованных констант

Перечисление называется Season и имеет четыре элемента

```
enum Season { Winter, Spring, Summer, Autumn };
let current: Season = Season.Summer;
console.log(current);       // 2
current = Season.Autumn;    // изменение значения
```

Созданное перечисление фактически эквивалентно следующему:

```
enum Season { Winter=0, Spring=1, Summer=2, Autumn=3 };
```

Можно задать значение одной константы, тогда значения следующих констант будет увеличиваться на единицу:

```
enum Season { Winter=5, Spring, Summer, Autumn };          // 5, 6, 7, 8
```

Можно каждой константе задать свое значение:

```
enum Season { Winter=4, Spring=8, Summer=16, Autumn=32 }; // 4, 8, 16, 32
```

Можно получить непосредственно текстовое значение:

```
enum Season { Winter=0, Spring=1, Summer=2, Autumn=3 };
var current: string = Season[2];    // 2 - числовое значение Summer
console.log(current);   // Summer
```

Строковые перечисления

В строковых перечислениях константы принимают строковые значения:

```
enum Season {
    Winter = "Зима",
    Spring = "Весна",
    Summer = "Лето",
    Autumn = "Осень"
};
var current: Season = Season.Summer;
console.log(current);   // Лето
```

Смешанные перечисления

Также можно определять смешанные перечисления, константы которых могут числа и строки.

```
enum Season {
    Winter = 1,
    Spring = "Весна",
    Summer = 3,
    Autumn = "Осень"
};
var current: Season = Season.Summer;
console.log(current);          // 3
console.log(Season.Autumn);    // Осень
```

Классы

```
class User {}
```

Здесь определено два объекта класса User - tom и alice:

```
let tom: User = new User();
let alice = new User();
```

Поля класса

При определении полей им можно задать некоторые начальные значения:

```
class User {
    name: string = "Tom Smith";
    age: number = 18;
}

let user = new User();
console.log(`name: ${user.name}  age: ${user.age}`);    // name: Tom Smith  age: 18
```

Методы

```
class User {
    name: string;
    age: number;

    print(){
        console.log(`name: ${this.name}  age: ${this.age}`);
    }

    toString(): string{
        return `${this.name}: ${this.age}`;
    }
}
```

Поля для чтения

Значение полей для чтения можно установить либо при объявлении, либо в конструкторе:

```
class User {

    readonly name: string = "Default user";
    age: number;

    constructor(userName: string, userAge: number) {
        this.name = userName;
        this.age = userAge;
    }

    print(){
        console.log(`name: ${this.name}  age: ${this.age}`);
    }
}
```

Абстрактные классы

Абстрактные классы представляют классы, определенные с ключевым словом abstract.

```
abstract class Figure {}
// let someFigure = new Figure()    // Ошибка!
```

```
abstract class Figure {
    getArea(): void{
        console.log("Not Implemented")
    }
}
class Rectangle extends Figure{
    constructor(public width: number, public height: number){
        super();
    }

    getArea(): void{
        let square = this.width * this.height;
        console.log("area =", square);
    }
}

let someFigure: Figure = new Rectangle(20, 30)
someFigure.getArea();   // area = 600
```

Абстрактные методы

```
abstract class Figure {
   abstract getArea(): void;
}
class Rectangle extends Figure{

   constructor(public width: number, public height: number){
      super();
   }

   getArea(): void{
      let square = this.width * this.height;
      console.log("area =", square);
   }
}

let someFigure: Figure = new Rectangle(20, 30)
someFigure.getArea();
```

Абстрактные поля

```
abstract class Figure {
    abstract x: number;
    abstract y: number;
    abstract getArea(): void;
}
class Rectangle extends Figure{
    //x: number;
    //y: number;

    constructor(public x: number, public y: number, public width: number,
public height: number){
        super();
    }

    getArea(): void{
        let square = this.width * this.height;
        console.log("area =", square);
    }
}

let someFigure: Figure = new Rectangle(10, 10, 20, 25)
someFigure.getArea();
```

Модификаторы доступа

В TypeScript три модификатора: public, protected и private.

```
class Person {

   private _name: string;
   private _year: number;
…
}
```

```
class Person {
   protected name: string;
   private year: number;
…
}
```

Используя модификаторы в параметрах конструктора, больше не надо явно создать свойства для этих параметров.

```
class Person {

   constructor(private name: string, private age: number) {  }

   printPerson(): void {
      console.log(`Имя: ${this.name}  Возраст: ${this.age}`);
   }
}
```

Интерфейсы объектов

Интерфейс - это определение кастомного типа данных, но без реализации.

```
interface IUser {
    id: number;
    name: string;
}
```

Теперь используем его в программе (employee - объект типа IUser):

```
let employee: IUser = {

    id: 1,
    name: "Tom"
}
console.log("id: ", employee.id);
console.log("name: ", employee.name);
```

Кроме свойств интерфейсы могут определять функции:

```
interface IUser {
    id: number;
    name: string;
    sayWords(words: string): void;
}
let employee: IUser = {

    id: 1,
    name: "Alice",
    sayWords: function(words: string): void{
        console.log(`${name} говорит "${words}"`);
    }
}

employee.sayWords("Привет, как дела?");
```

Интерфейсы классов

Интерфейсы могут быть реализованы не только объектами, но и классами. Для этого используется ключевое слово implements:

```
interface IUser {
    id: number;
    name: string;
    getFullName(surname: string): string;
}

class User implements IUser{
    id: number;
    name: string;
    age: number;
    constructor(userId: number, userName: string, userAge: number) {
        this.id = userId;
        this.name = userName;
        this.age = userAge;
    }
    getFullName(surname: string): string {

        return this.name + " " + surname;
    }
}

let tom = new User(1, "Tom", 23);
console.log(tom.getFullName("Simpson"));
```

Расширение интерфейса

TypeScript позволяет добавлять в интерфейс новые поля и методы, просто объявив интерфейс с тем же именем и определив в нем необходимые поля и методы. В итоге объект или класс, который реализует этот интерфейс, должен определить все три поля (id, name и age):

```
interface IUser {
    id: number;
    name: string;
}
interface IUser{
    age: number;
}
let employee: IUser = {
    id: 1,
    name: "Alice",
    age: 31
}

function printUser(user: IUser): void {
    console.log(`id: ${user.id}  name: ${user.name}  age: ${user.age}`);
}

printUser(employee);
```

Интерфейсы, как и классы, могут наследоваться:

```
interface IMovable {

   speed: number;
   move(): void;
}
interface ICar extends IMovable {

   fill(): void;
}
```

Гибридные интерфейсы

Интерфейсы могут сочетать различные стили, могут применяться сразу как к определению объекта, так и к определению функции:

```
interface PersonInfo {
   (name: string, surname: string):void;
   fullName: string;
   password: string;
   authenticate(): void;
}
```

Обобщения

В этом случае мы не можем использовать результат функции как объект того типа, который передан в функцию, так как функция возвращает тип any.

```
function getId(id: any): any {

    return id;
}
let result = getId(5);
console.log(result);
```

Чтобы конкретизировать возвращаемый тип, можно использовать обобщения:

```
function getId<T>(id: T): T {
    return id;
}

let result1 = getId<number>(5);
console.log(result1);
let result2 = getId<string>("abc");
console.log(result2);
```

```
function getString<T>(args: Array<T>): string {
    return args.join(“, ”);
}

let result1 = getString <number>([1, 2, 3]);
console.log(result1);
```

Обобщенные классы

```
class User<T> {

    private _id: T;
    constructor(id:T) {

        this._id=id;
    }
    getId(): T {

        return this._id;
    }
}

let tom = new User<number>(3);
console.log(tom.getId()); // возвращает number

let alice = new User<string>("vsf");
console.log(alice.getId()); // возвращает string
```

Надо учитывать, что если объект типизирован, то сменить данный тип уже не получится. То есть в коде ниже второе создание объекта не будет работать, так как объект tom уже типизирован типом number:

```
let tom = new User<number>(3);
console.log(tom.getId());
tom = new User<string>("vsf"); // ошибка
```

Ограничения обобщений

Ограничения или constraints позволяют ограничить набор типов, которые могут использоваться в обобщениях.

<T extends {name:string}> означает, что параметр T должен представлять такой тип, который содержит свойство name

```
function compareName<T extends {name:string}>(obj1: T, obj2: T): void{

    if(obj1.name === obj2.name){
        console.log("Имена совпадают");
    }
    else{
        console.log("Имена различаются");
    }
}

let tom: {name:string} = {name: "Tom"};
let sam: {name: string} = {name: "Sam"};
compareName<{name:string}>(tom, sam);
```

Ключевое слово new

Чтобы создать новый объект в коде обобщений, надо указать, что обобщенный тип T имеет конструктор -- вместо параметра type:T надо указать type: {new(): T;}:

```
function UserFactory<T>(): T {
    return new T(); // ошибка компиляции
}
```

Чтобы интерфейс начал работать, используем слово new:

```
function userFactory<T>(type: { new (): T; }): T {
    return new type();
}
```

```
class User {
    constructor() {
        console.log("создан объект User");
    }
}

let user : User = userFactory(User);
```

Пространства имен

Для определения пространств имен используется ключевое слово **namespace**. Чтобы типы и объекты, определенные в пространстве имен, были видны извне, они определяются с ключевым словом **export**.

```
namespace Personnel {
    export class Employee {
        constructor(public name: string){
        }
    }
}

let alice = new Personnel.Employee("Alice");
console.log(alice.name);    // Alice
```

Вложенные пространства имен

```
namespace Data{
    export namespace Personnel {
        export class Employee {
            constructor(public name: string){
            }
        }
    }
    export namespace Clients {
        export class VipClient {
            constructor(public name: string){
            }
        }
    }
}

let tom = new Data.Personnel.Employee("Tom")
console.log(tom.name);

let sam = new Data.Clients.VipClient("Sam");
console.log(sam.name);
```

Псевдонимы

Псевдонимы, задаются с помощью ключевого слова import:

```
namespace Data{
   export namespace Personnel {
      export class Employee {

         constructor(public name: string){
         }
      }
   }
}

import employee = Data.Personnel.Employee;
let tom = new employee("Tom")
console.log(tom.name);
```

```
export default function hello() {
  console.log("Hello Typescript");
}
```

```
import hello from "./message.js";
hello();
```

Компиляция модуля

При компиляции из командной строки или терминала для установки модуля необходимо передать соответствующее значение параметру --module:

```
tsc --module commonjs main.ts // для CommonJS
```

Для компиляции модулей надо ввести команду:

```
tsc main.ts --module esnext
```

После этого в папке появятся файлы main.js

Вместо того, чтобы указывать тип модуля в консоли при компиляции, можно использовать соответствующие параметры в файле конфигурации tsconfig.json. Так, параметр module задает тип модуля:

```
{
    "compilerOptions": {
        "noImplicitAny": true,
        "noEmitOnError": true,
        "strictNullChecks": true,
        "outFile": "main.js"

        "target": "es2015",
        "module": "esnext"
    }
}
```

Заголовочные файлы

Декларативные или заголовочные файлы служат для установки связи с внешними файлами скриптов javascript в TS, имеют расширение .d.ts. Они описывают синтаксис и структуру функций и свойств, которые могут использоваться в программе, не предоставляя при этом конкретной реализации.

```
<!DOCTYPE html>
<html>
<head>
   <meta charset="utf-8" />
   <title>Metanit.com</title>
</head>
<body>
   <h2>Приложение на TypeScript</h2>
   <script>
      let message = "Hello TypeScript!";
   </script>
   <script src="app.js"></script>
</body>
</html>
```

С помощью ключевого слова declare в программу на TS подключается определение глобальной переменной в файле globals.d.ts :

```
declare let message: string;
```

Получится следующая структура проекта:

- app.ts
- globals.d.ts
- index.html

В файле app.ts надо явно указать расположение файла globals.d.ts с помощью директивы reference, чтобы компилятор увидел файл globals.d.ts

```
<reference path="globals.d.ts" />
console.log(message);
```

Заголовочные файлы для популярных библиотек

Сообщество TypeScript создало общий репозиторий для декларативных заголовочных файлов популярных библиотек (jquery, angularjs, extjs и др.), чтобы не надо было заново определять свои файлы, а можно бы было взять уже готовые. Этот репозиторий расположен на гитхабе:

https://github.com/DefinitelyTyped/DefinitelyTyped/

Использование js-библиотеки и заголовочных файлы рассмотрим на примере библиотеки jquery. Вначале загрузим определения типов для библиотеки jquery с помощью следующей команды:

```
npm install --save-dev @types/jquery
```

В итоге в проекте будет создана папка node_modules/@types, в которой каталог jquery будет хранить заголовочные файлы для библиотеки jquery. Кроме того, в проекте будет создан файл package.json со следующим содержимым:

```
{
  "devDependencies": {
    "@types/jquery": "^3.5.5"
  }
}
```

Надо указать компилятору TypeScript с помощью параметра typeRoots, где находятся заголовочные файлы -- это каталог node_modules/@types. Для этого в проекте определим следующий файл конфигурации tsconfig.json:

```
{
   "compilerOptions": {
      "noImplicitAny": true,
      "noEmitOnError": true,
      "strictNullChecks": true,

      "outFile": "app.js",

      "typeRoots": [
        "node_modules/@types"
      ],
   }
}
```

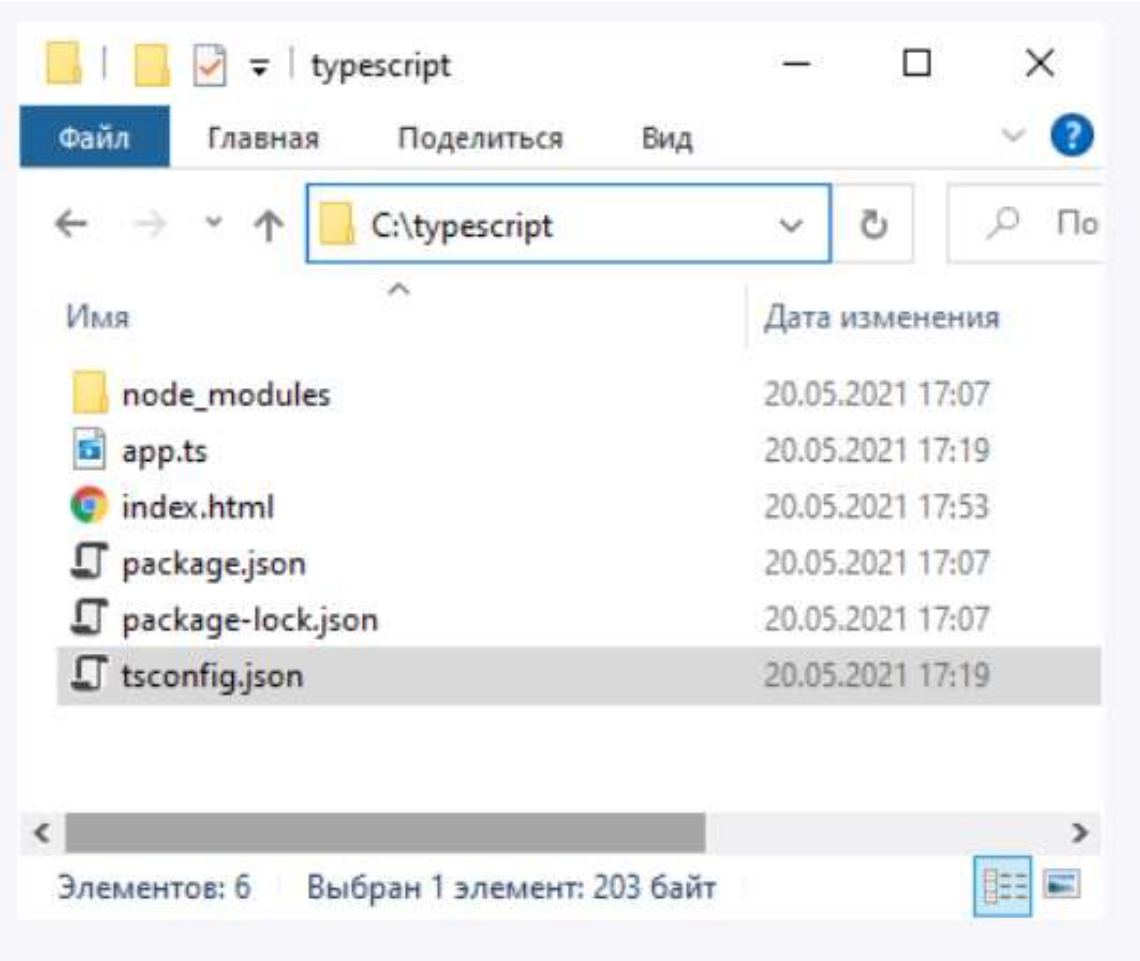
В корневой папке проекта определим веб-страницу index.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Metanit.com</title>
</head>
<body>
    <div id="content"></div>
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
    <script src="app.js"></script>
</body>
</html>
```

и файла app.ts

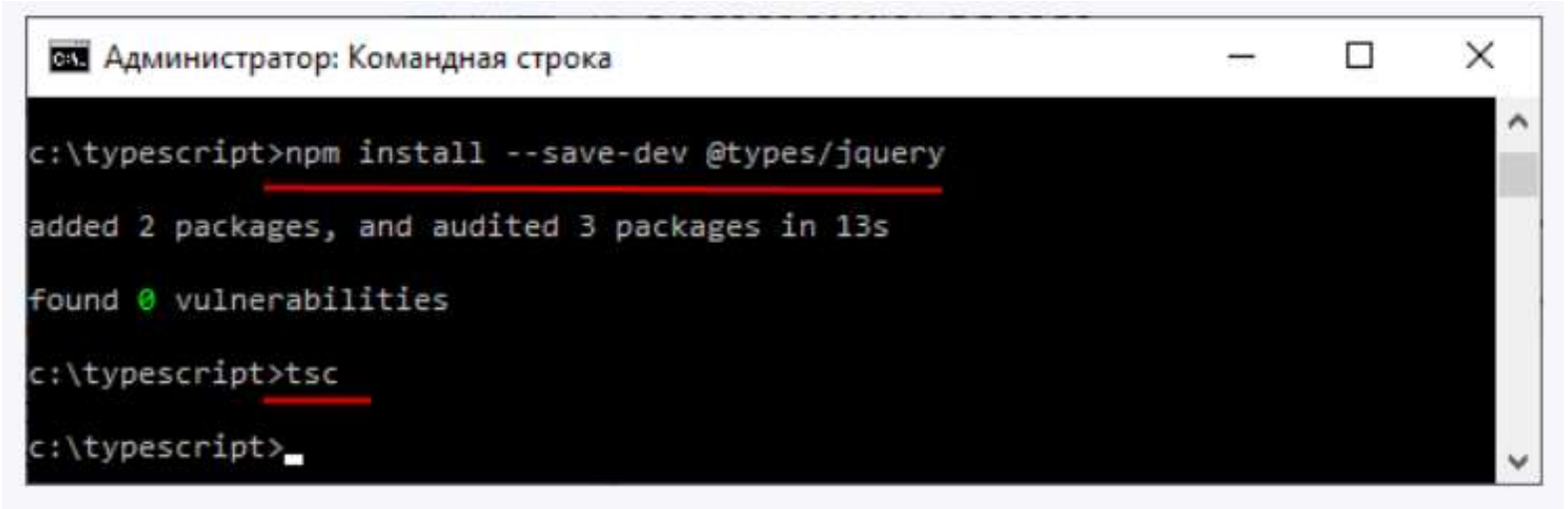
```
$(document).ready(() => {
    $("#content").html("<h1>Привет TypeScript</h1>");
});
```

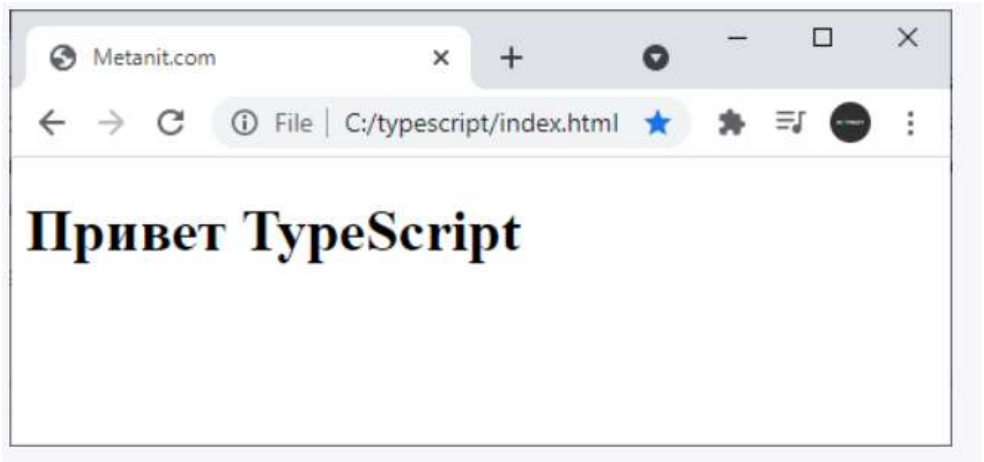
В итоге проект будет выглядеть следующим образом:



Скомпилируем файл приложения с помощью команды

tsc





Декораторы

Декораторы являются инструментом декларативного программирования, они позволяют добавить к классам и их членам метаданные и тем самым изменить их поведение без изменения их кода.

Декоратор sealed с помощью функции Object.seal запрещает расширение прототипа класса User. Для применения декоратора используется знак @ :

```
function sealed(constructor: Function) {
    console.log("sealed decorator");
    Object.seal(constructor);
    Object.seal(constructor.prototype);
}

@sealed
class User {
    name: string;
    constructor(name: string){
        this.name = name;
    }
    print():void{
        console.log(this.name);
    }
}
```